

# 랜덤 터레인 디펜스 작업물 리포트

2026 넥슨 대학생 게임잼 '재밋넥' 2차 과제 전형

|          |                                    |
|----------|------------------------------------|
| 제작자      | 김영목                                |
| 과제       | 2026 넥슨 대학생 게임잼 '재밋넥' 2차 과제 전형     |
| 주제       | 자신만의 탑뷰 디펜스 게임                     |
| 개발 언어    | C++20                              |
| 그래픽 API  | DirectX 12(D3D12)                  |
| 개발 방식    | DirectX12 참고 프레임워크 기반 커스텀 확장       |
| 참고 프레임워크 | SH4MDEL/DirectX12-ComputerGraphics |
| 작품명      | 랜덤 터레인 디펜스                         |
| 작성일      | 2026.07.06                         |

본 리포트는 결과물 설명, 제작 과정, AI 활용 내역, 제작 중 어려웠던 점과 해결 방법을 제출 형식에 맞춰 정리한 문서입니다.

## 1. 결과물에 대한 설명

랜덤 터레인 디펜스는 마우스로 타워를 구매하고 지형 위에 배치하여, 정해진 경로를 따라 이동하는 적을 막는 탑류 디펜스 게임입니다. 기본적인 타워 디펜스 구조에 랜덤 상점, 타워 합성, 지형 기반 배치, 공중 몬스터와 대공 타워, 스펠 카드, 보스 보상 선택을 결합했습니다.

게임은 메인 화면에서 스테이지와 난이도를 선택한 뒤 시작됩니다. 플레이어는 하단 상점에서 랜덤으로 등장하는 타워나 스펠을 드래그하여 사용하고, 같은 종류와 같은 성급의 타워를 합성해 더 높은 성급의 타워를 만들 수 있습니다. 웨이브가 끝나면 보상 카드를 선택하고, 보스 처치 시 별도의 보상 선택지가 표시됩니다.

주요 플레이 흐름은 다음과 같습니다.

- 메인 화면에서 스테이지와 난이도 선택
- 상점에서 타워, 스펠, 코인 생성기 구매
- 마우스 드래그로 지형 위에 배치
- 같은 종류와 성급의 타워를 합성
- 웨이브 시작 후 3갈래 경로에서 등장하는 적 방어
- 웨이브 보상과 보스 보상을 선택하며 성장
- 최종 웨이브까지 생명을 지키면 승리

구현된 주요 콘텐츠는 다음과 같습니다.

- 타워 종류: 기본, 연사, 폭발, 빙결, 박격, 대공
- 스펠 종류: 운석, 감속, 강화, 바위 굴리기
- 경제 요소: 코인, 랜덤 상점, R키 리롤, 코인 생성기
- 성장 요소: 1성-3성 타워, 합성, 타워 종류별 데미지 강화
- 적 종류: 일반, 빠른 적, 중장갑, 대형 적, 공중 적, 분열 공중 적, 보스
- 스테이지: 초원, 능선, 원형 경로
- 난이도: 쉬움, 보통, 어려움
- UI: 상점 접기/펼치기, 미니맵, 타워 정보창, 강화 UI, 웨이브/보스 보상 UI, 결과 화면
- 카메라: 탑류와 각도뷰 전환, WASD 이동, 마우스 휠 확대/축소, 디버그 카메라
- 연출: 파티클, 데미지 숫자 팝업, 충격 링, 보스 등장 연출, 낮과 밤이 있는 태양/달 조명
- 사운드: 클릭, 배치, 리롤, 코인 획득, 피격, 스펠, 웨이브 시작, 보스 등장 효과음

기술적으로는 C++20과 DirectX 12(D3D12)를 사용했습니다. 프레임워크는 처음부터 모든 코드를 새로 작성한 완전 자체 엔진이 아니라, SH4MDEL/DirectX12-ComputerGraphics와 티스토리 DirectX12 학습 자료를 참고한 기존 구조를 과제 목적에 맞게 수정하고 확장한 형태입니다. 그 위에 Scene, GameObject, Material, Collider, Rigidbody, TerrainCollider, Shadow, Camera, Mesh 등의 구조를 정리하여 사용했습니다. Unity에서 추출한 계층형 터렛 모델은 실행 중 FBX를 직접 읽는 방식이 아니라, 미리 자체 바이너리 포맷으로 변환하여 로드하도록 구성했습니다. 이를 통해 실행 중 파싱 비용을 줄이고, 터렛의 베이스와 회전부를 분리하여 공격 대상 방향으로 상단만 회전하도록 구현했습니다.

Unity나 Unreal Engine 같은 상용 엔진을 사용하지 않은 이유는, 이번 과제에서 단순히 에디터 기능을 조립하는 것보다 렌더링, 충돌, 터레인, 카메라, UI, 에셋 로딩 파이프라인을 코드 단위로 직접 이해하고 연결하는 제작 역량을 보여주고 싶었기 때문입니다. 또한 이미 가지고 있던 DirectX12 기반 프레임워크를 수정/확장하는 방식이 짧은 과제 기간 안에서 변경 범위가 명확했고, AI와 협업할 때도 대규모 엔진 설정과 에디터 작업을 설명하는 것보다 필요한 코드 단위만 다루는 편이 시간과 토큰 사용량을 줄여 빠른 반복에 유리했습니다.

## 2. 제작 과정

### 2.1 기존 프레임워크 정리와 씬 구성

처음에는 기존 프레임워크를 복사해 온 상태였고, 게임잼 과제에 맞게 불필요한 플레이어 오브젝트와 기존 테스트 오브젝트를 제거했습니다. 이후 탑류 디펜스 전용 씬을 만들고, 메인 화면과 인게임 화면을 구분했습니다. SceneManager를 통해 ESC 입력 시 메뉴로 돌아가거나 마지막 씬에서는 게임을 종료하도록 정리했습니다.

## 2.2 상용 엔진 대신 DirectX12 참고 프레임워크를 확장한 이유

Unity는 빠른 프로토타이핑에 강점이 있지만, 이번 결과물에서는 C++/DirectX 기반 구조 위에서 게임 제작 전체 흐름을 주도했다는 점을 보여주는 것이 더 의미 있다고 판단했습니다. 다만 프레임워크 자체는 완전히 무에서 새로 작성한 것이 아니라, SH4MDEL/DirectX12-ComputerGraphics와 티스토리 DirectX12 학습 자료를 참고한 기존 구조를 기반으로 했습니다. Unity는 모델 에셋을 준비하고 내보내는 도구로만 사용하고, 실제 게임 실행, 렌더링, 충돌 처리, UI, 웨이브 로직은 해당 DirectX12 기반 프레임워크를 수정/확장하여 구현했습니다.

이 선택은 AI Native 과제 진행 방식과도 잘 맞았습니다. 상용 엔진 프로젝트를 새로 만들면 씬 설정, 패키지 버전, 에디터 상태, 프리팹 연결 같은 맥락을 계속 설명해야 합니다. 반면 기존 C++ 코드베이스를 확장하면 AI에게 필요한 함수와 파일 단위로 요청할 수 있어 토큰과 시간을 아끼면서도 결과물을 빠르게 검증할 수 있었습니다. 또한 참고 프레임워크 위에서 DirectX 렌더링, 충돌 판정, 터레인 좌표 변환, 계층형 모델 로딩 같은 문제를 직접 수정하며 기술적 판단을 보여줄 수 있었습니다.

## 2.3 터레인 기반 디펜스 맵 구현

초기에는 격자 기반 배치를 사용했지만, 과제의 완성도를 높이기 위해 터레인 위에서 실제 방어하는 구조로 변경했습니다. TerrainHeightMap과 TerrainCollider를 이용해 마우스 위치를 지형 위 좌표로 변환하고, 타워 배치 가능 여부를 지형 경사와 경로 충돌 기준으로 판정했습니다.

스테이지별로 다른 느낌을 주기 위해 지형과 경로를 분리해서 설계했습니다. 초원은 기본형, 능선은 경사가 강한 지형, 원형은 안쪽 공간과 외곽 경로가 구분되는 구조로 만들었습니다. 적이 길에서 끼지 않도록 경로 폭과 경사, 높이 차이를 반복적으로 조정했습니다.

## 2.4 상점, 랜덤성, 합성 시스템

랜덤 디펜스 장르의 핵심은 매번 다른 선택지가 주어지는 점이라고 판단했습니다. 그래서 상점에는 항상 랜덤한 3개의 상품이 등장하고, 상품은 타워뿐 아니라 스펠과 코인 생성기도 포함되도록 했습니다. 플레이어는 R키로 코인을 사용해 상점을 리롤할 수 있습니다.

타워는 1성부터 3성까지 존재하며, 같은 종류와 같은 성급의 타워를 드래그해서 합칠 수 있습니다. 상점에서 바로 같은 타워 위로 드래그해 합성할 수도 있고, 이미 배치된 타워끼리도 드래그해서 합성할 수 있습니다. 합성 가능 상태가 보이도록 강조 표시도 추가했습니다.

## 2.5 적, 웨이브, 보스 구성

적은 3개의 경로에서 등장해 중간 지점으로 합류하도록 구성했습니다. 웨이브가 진행될수록 일반 적만 반복되지 않도록 빠른 적, 대형 적, 중장갑 적, 공중 적, 분열 공중 적, 보스가 섞여 나오게 했습니다. 공중 적은 일반 타워가 공격하지 못하고, 대공 타워가 전담하도록 타겟 판정을 분리했습니다.

보스 웨이브에서는 카메라 흔들림과 등장 파티클, 별도 효과음을 넣어 일반 웨이브와 차별화했습니다. 보스를 처치하면 보스 선택 UI가 표시되고, 일반 웨이브 종료 시에도 성장 선택지를 제공하여 플레이 흐름이 끊기지 않도록 했습니다.

## 2.6 UI와 조작 개선

초기 UI는 기능 확인용에 가까웠기 때문에, 실제 게임처럼 보이도록 상단 HUD, 하단 상점, 좌하단 강화 UI, 우상단 타워 정보 UI, 우하단 미니맵으로 역할을 분리했습니다. 상점은 버튼으로 접고 펼칠 수 있고, 코인과 생명은 한눈에 볼 수 있도록 상단 왼쪽에 배치했습니다.

또한 타워를 클릭하면 공격력, 사거리, 공격 속도, 쿨다운 정보를 표시하고, 정보창 안에 타워 모델이 위쪽 옆면에서 보이도록 조정했습니다. 한글 UI가 깨지거나 사라지는 문제는 폰트 렌더링 방식을 보완하고, 필요한 경우 Malgun Gothic 폰트로 fallback하도록 해결했습니다.

## 2.7 모델, 재질, 사운드, 연출 추가

터렛 모델은 Unity Asset Store의 WGX3D Low Poly Turret Pack 5 모델 에셋을 사용했습니다. Unity에서 계층 구조를 유지한 채 내보낸 뒤, 자체 엔진용 바이너리 포맷으로 변환했습니다. 모델 파트별 매니페스트를 사용하여 회전해야 하는 부품과 고정되어야 하는 부품을 구분했습니다. 재질 클래스와 텍스처 로딩 구조도 추가하여 단색 큐브만 보이는 상태에서 벗어나 모델과 텍스처를 사용할 수 있게 했습니다.

사운드는 Kenney의 CC0 무료 UI/효과음 에셋을 사용했습니다. 클릭, 상점 리롤, 타워 배치, 코인 획득, 타격, 스펠 사용, 웨이브 시작, 보스 등장 등 플레이어가 행동을 확인해야 하는 순간에 사운드를 배치했습니다. 전투 중 피격음은 너무 자주 겹치지 않도록 짧은 쿨다운을 적용했습니다.

### 3. AI 활용 내역

이번 과제는 AI Native 게임잼의 취지에 맞춰 AI를 적극적으로 활용했습니다. 사용 목적은 단순 코드 자동 생성이 아니라, 짧은 기간 안에 기획, 구현, 디버깅, 개선 방향 판단을 빠르게 반복하기 위한 협업 도구로 활용하는 것이었습니다.

활용한 AI와 용도는 다음과 같습니다.

- OpenAI ChatGPT/Codex: 게임 구조 기획, C++ 코드 구현 보조, 버그 원인 분석, UI 개선 방향 제안, 시스템 리팩터링, 빌드 오류 해결, 보고서 작성 보조
- AI 기반 코드 협업: Scene, GameObject, Material, Collider, Terrain, Shadow 등 기존 프레임워크에 맞는 구현 방향을 정리하고, SOLID 원칙과 모던 C++ 스타일에 맞게 책임을 나누는 데 활용
- AI 기반 기술 선택 정리: 상용 엔진을 사용하지 않고 C++20/DirectX 12 참고 프레임워크를 수정/확장하는 이유를 정리하고, 짧은 기간 안에서 변경 범위와 AI 토큰 사용량을 줄이는 개발 방식으로 구체화하는 데 활용
- AI 기반 디버깅: 마우스 좌표와 카메라 레이 불일치, 터레인 경사로 인한 적 이동 문제, 계층형 모델 회전 문제, 한글 UI 폰트 문제 등을 빠르게 추적하는 데 활용
- AI 기반 콘텐츠 확장: 랜덤 상점, 타워 합성, 보스 보상, 웨이브 보상, 공중 몬스터, 박격포, 스펠, 미니맵, 사운드 피드백 등 추가 기능 후보를 정리하는 데 활용
- AI 기반 문서화: 제작 과정을 제출용 리포트 구조로 정리하고, 구현 의도와 문제 해결 과정을 설명하는 데 활용

AI가 제안한 내용은 그대로 적용하지 않고, 현재 프레임워크 구조와 과제 일정에 맞게 필요한 부분만 선택했습니다. 구현 후에는 직접 빌드하고 동작을 확인하면서 문제를 수정했습니다.

### 4. 어려웠던 점과 해결 방법

#### 4.1 마우스 좌표와 실제 배치 위치 불일치

상점에서 드래그한 타워가 마우스 위치와 다르게 잡히거나, 카메라 시점에 따라 배치 지점이 어긋나는 문제가 있었습니다. 원인은 화면 좌표, 카메라 레이, 지형 충돌 판정이 서로 다른 기준으로 계산되고 있었기 때문입니다. 이를 해결하기 위해 카메라의 U, V, N 벡터와 FOV를 기준으로 마우스 레이를 만들고, TerrainCollider와 교차한 지점을 배치 기준으로 통일했습니다.

#### 4.2 터레인 경사에서 적이 끼는 문제

지형을 산처럼 만들수록 적이 경로에서 끼거나 경사를 오르지 못하는 문제가 발생했습니다. 단순히 이동 속도를 올리는 방식은 부자연스러웠기 때문에, 경로 주변의 지형 높이를 낮추고 경사 차이를 줄였습니다. 동시에 길 폭을 넓히고 웨이포인트를 더 촘촘하게 배치하여 적 이동이 안정적으로 보이도록 했습니다.

#### 4.3 계층형 터렛 모델의 부분 회전

처음에는 공격 시 타워 전체가 회전에 베이스까지 같이 돌아가는 문제가 있었습니다. Unity 모델의 계층 구조를 그대로 활용하기 위해 모델 파트를 매니페스트로 저장하고, rotatesWithTarget 값을 가진 파트만 타겟 방향으로 회전하도록 수정했습니다. 그 결과 베이스는 고정되고 터렛 상단만 공격 방향을 향하게 되었습니다.

#### 4.4 한글 UI와 비트맵 폰트 문제

UI를 한국어로 바꾼 뒤 별 표시나 보상 카드 텍스트가 일부 사라지는 문제가 있었습니다. 기존 비트맵 폰트가 한글 글리프를 포함하지 않아 발생한 문제였고, 한글 문자열은 Malgun Gothic fallback으로 렌더링하도록 처리했습니다. 또한 텍스트 오브젝트 풀 크기를 늘려 보상 UI와 결과 화면에서도 글자가 사라지지 않게 했습니다.

#### 4.5 짧은 일정에서 기능 우선순위 정하기

제한된 기간 안에 모든 기능을 완성도 높게 구현하기는 어려웠습니다. 그래서 우선순위를 핵심 플레이 루프, 조작감, 제출 시연에서 보이는 완성도 순서로 정했습니다. 먼저 배치, 공격, 웨이브, 보상 루프를 안정화했고, 이후 UI, 모델, 사운드, 연출을 추가했습니다.

## 5. 외부 에셋 및 라이선스

사운드 에셋은 Kenney의 UI Audio와 Interface Sounds를 사용했습니다. 해당 에셋은 Creative Commons Zero, CC0 라이선스로 개인, 교육, 상업 프로젝트에 사용할 수 있으며 크레딧 표기는 필수가 아닙니다. 프로젝트에는 출처와 라이선스를 기록한 AUDIO\_CREDITS.txt 파일을 함께 포함했습니다.

터렛 모델은 Unity Asset Store의 WGX3D Low Poly Turret Pack 5 에셋을 사용했습니다. 사용 용도는 타워 모델과 관련 텍스처이며, Unity에서 내보낸 뒤 자체 엔진용 바이너리 포맷으로 변환하여 사용했습니다. 해당 에셋은 Unity Asset Store에서 제공되는 에셋이므로 Unity Asset Store Terms of Service and EULA 기준으로 사용 범위를 확인했습니다. 외부 에셋 사용 시 라이선스 범위를 확인해야 한다는 제출 안내에 맞춰, 사용한 에셋은 별도 출처와 라이선스 확인이 가능한 형태로 관리했습니다.

프레임워크는 SH4MDEL/DirectX12-ComputerGraphics 저장소와 티스토리 DirectX12 학습 자료를 참고한 구조를 기반으로 하되, 이번 과제에서는 탐류 디펜스 게임에 필요한 씬 구성, 터레인 경로, 물리/충돌 처리, UI, 그림자, 모델 바이너리 로더, 상점과 웨이브 시스템을 직접 수정하고 확장했습니다.

- 참고 프레임워크: SH4MDEL/DirectX12-ComputerGraphics

<https://github.com/SH4MDEL/DirectX12-ComputerGraphics>

- Unity Asset Store 모델 에셋: WGX3D Low Poly Turret Pack 5

<https://assetstore.unity.com/packages/3d/props/guns/wgx3d-low-poly-turret-pack-5-384604>

- Unity Asset Store Terms of Service and EULA

<https://unity.com/legal/as-terms>

## 6. 실행 및 검증

개발 환경은 Windows, Visual Studio C++ 프로젝트이며, 개발 언어는 C++20, 그래픽 API는 DirectX 12(D3D12)입니다. 프레임워크는 SH4MDEL/DirectX12-ComputerGraphics와 티스토리 DirectX12 학습 자료를 참고한 구조를 과제에 맞게 수정/확장했습니다. Debug x64 빌드 기준으로 실행 파일을 생성했습니다. 최종 확인 시 빌드는 경고 0개, 오류 0개로 성공했습니다.

기본 조작은 다음과 같습니다.

- 마우스: 상점 상품 드래그, 타워 배치, 타워 선택, 합성
- R: 상점 리롤
- V: 카메라 시점 전환
- WASD: 카메라 이동
- 마우스 휠: 확대/축소
- F: 게임 배속 전환
- B: 콜라이더 디버그 카메라
- ESC: 메뉴로 이동 또는 게임 종료

## 7. 아쉬운 점과 추가 개선 방향

현재 버전은 과제 기간 안에 완성 가능한 범위에서 핵심 루프와 시연용 완성도를 우선했습니다. 추가 시간이 있다면 다음 요소를 더 개선하고 싶습니다.

- 타워와 적의 밸런스 수치 세밀화
- 스테이지별 고유 기믹 추가
- 저장/불러오기와 기록 시스템

- 사운드 볼륨 믹서와 옵션 UI
- 더 다양한 모델과 애니메이션
- 튜토리얼과 초보자 안내 흐름

이번 작업을 통해 단순히 하나의 기능을 구현하는 것보다, 기획, 엔진 구조, UI, 에셋 파이프라인, 디버깅, 제출 문서화까지 전체 제작 과정을 끝까지 연결하는 것이 중요하다는 점을 확인했습니다.

## 추신. 별도 역량 참고자료

이번 과제 결과물에는 2D 캐릭터 드로잉 리소스를 직접 포함하지 않았지만, 개인적으로 2D 캐릭터 드로잉도 가능합니다. 게임 제작 과정에서 캐릭터 콘셉트나 러프 일러스트가 필요한 경우 직접 시각화하여 방향성을 잡을 수 있습니다. 해당 역량은 이번 게임의 필수 제출 항목이 아닌 별도 참고자료로만 제시합니다.

- 2D 캐릭터 드로잉 쇼츠 참고자료

<https://youtube.com/shorts/bNPkp19TjSA?si=uFguoGw4Ruoc4ly3>